

Chương 10 – Giới thiệu về Mạng Nơ-ron Nhân tạo với Keras

Sổ tay (notebook) này chứa tất cả mã mẫu và lời giải cho các bài tập trong chương 10.

 [Open in Colab](#)  [Open in Kaggle](#)

✓ Thiết lập

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Và TensorFlow ≥ 2.8 :

```
import tensorflow as tf

assert version.parse(tf.__version__) >= version.parse("2.8.0")
```

Như chúng ta đã làm trong các chương trước, hãy định nghĩa kích thước phông chữ mặc định để làm cho các hình vẽ đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục `images/ann` (nếu nó chưa tồn tại), và định nghĩa hàm `save_fig()` được sử dụng xuyên suốt sổ tay (notebook) này để lưu các hình vẽ ở độ phân giải cao cho sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "ann"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

✓ Từ Nơ-ron Sinh học đến Nơ-ron Nhân tạo

Perceptron

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
```

```
iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"].values
y = (iris.target == 0) # Iris setosa

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)

X_new = [[2, 0.5], [3, 1]]
y_pred = per_clf.predict(X_new) # predicts True and False for these 2 flowers
```

y_pred

array([True, False])

Perceptron tương đương với một `SGDClassifier` với `loss="perceptron"`, không có chuẩn hóa (regularization), và tốc độ học (learning rate) không đổi bằng 1:

```
# extra code - shows how to build and train a Perceptron

from sklearn.linear_model import SGDClassifier

sgd_clf = SGDClassifier(loss="perceptron", penalty=None,
                        learning_rate="constant", eta0=1, random_state=42)
sgd_clf.fit(X, y)
assert (sgd_clf.coef_ == per_clf.coef_).all()
assert (sgd_clf.intercept_ == per_clf.intercept_).all()
```

Khi Perceptron tìm thấy một **đường biên quyết định** (decision boundary) phân tách các lớp một cách thích hợp, nó sẽ ngừng học. Điều này có nghĩa là đường biên quyết định thường khá gần với một trong các lớp:

```
# extra code - plots the decision boundary of a Perceptron on the iris dataset

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

a = -per_clf.coef_[0, 0] / per_clf.coef_[0, 1]
b = -per_clf.intercept_ / per_clf.coef_[0, 1]
axes = [0, 5, 0, 2]
x0, x1 = np.meshgrid(
    np.linspace(axes[0], axes[1], 500).reshape(-1, 1),
    np.linspace(axes[2], axes[3], 200).reshape(-1, 1),
)
X_new = np.c_[x0.ravel(), x1.ravel()]
y_predict = per_clf.predict(X_new)
zz = y_predict.reshape(x0.shape)
custom_cmap = ListedColormap(['#9898ff', '#fafab0'])

plt.figure(figsize=(7, 3))
plt.plot(X[y == 0, 0], X[y == 0, 1], "bs", label="Not Iris setosa")
plt.plot(X[y == 1, 0], X[y == 1, 1], "yo", label="Iris setosa")
plt.plot([axes[0], axes[1]], [a * axes[0] + b, a * axes[1] + b], "k-",
         linewidth=3)
plt.contourf(x0, x1, zz, cmap=custom_cmap)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="lower right")
plt.axis(axes)
plt.show()
```

Hàm kích hoạt

```
# extra code - this cell generates and saves Figure 10-8

from scipy.special import expit as sigmoid

def relu(z):
    return np.maximum(0, z)

def derivative(f, z, eps=0.000001):
    return (f(z + eps) - f(z - eps))/(2 * eps)

max_z = 4.5
z = np.linspace(-max_z, max_z, 200)

plt.figure(figsize=(11, 3.1))

plt.subplot(121)
plt.plot([-max_z, 0], [0, 0], "r-", linewidth=2, label="Heaviside")
plt.plot(z, relu(z), "m-", linewidth=2, label="ReLU")
plt.plot([0, 0], [0, 1], "r-", linewidth=0.5)
plt.plot([0, max_z], [1, 1], "r-", linewidth=2)
plt.plot(z, sigmoid(z), "g--", linewidth=2, label="Sigmoid")
plt.plot(z, np.tanh(z), "b-", linewidth=1, label="Tanh")
plt.grid(True)
plt.title("Activation functions")
plt.axis([-max_z, max_z, -1.65, 2.4])
plt.gca().set_yticks([-1, 0, 1, 2])
plt.legend(loc="lower right", fontsize=13)

plt.subplot(122)
plt.plot(z, derivative(np.sign, z), "r-", linewidth=2, label="Heaviside")
plt.plot(0, 0, "ro", markersize=5)
plt.plot(0, 0, "rx", markersize=10)
plt.plot(z, derivative(sigmoid, z), "g--", linewidth=2, label="Sigmoid")
plt.plot(z, derivative(np.tanh, z), "b-", linewidth=1, label="Tanh")
plt.plot([-max_z, 0], [0, 0], "m-", linewidth=2)
plt.plot([0, max_z], [1, 1], "m-", linewidth=2)
plt.plot([0, 0], [0, 1], "m-", linewidth=1.2)
plt.plot(0, 1, "mo", markersize=5)
plt.plot(0, 1, "mx", markersize=10)
plt.grid(True)
plt.title("Derivatives")
plt.axis([-max_z, max_z, -0.2, 1.2])

save_fig("activation_functions_plot")
plt.show()
```

▼ MLP Hồi quy

```
from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
```

```

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np

# Tải dữ liệu California Housing
housing = fetch_california_housing()

# Chia dữ liệu train / test / validation
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target, random_state=42
)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42
)

# Khởi tạo mô hình MLP với 3 tầng ẩn (50 nơon mỗi tầng)
mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], random_state=42)

# Tạo pipeline với StandardScaler để chuẩn hóa dữ liệu đầu vào
pipeline = make_pipeline(StandardScaler(), mlp_reg)

# Huấn luyện mô hình
pipeline.fit(X_train, y_train)

# Dự đoán trên tập validation
y_pred = pipeline.predict(X_valid)

# Tính RMSE (căn bậc hai của MSE)
mse = mean_squared_error(y_valid, y_pred)
rmse = np.sqrt(mse)

print("RMSE:", rmse)

```

RMSE: 0.5053326657968684

rmse

np.float64(0.5053326657968684)

▼ MLP Phân loại

```

# extra code - this was left as an exercise for the reader

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

iris = load_iris()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    iris.data, iris.target, test_size=0.1, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, test_size=0.1, random_state=42)

mlp_clf = MLPClassifier(hidden_layer_sizes=[5], max_iter=10_000,
                        random_state=42)
pipeline = make_pipeline(StandardScaler(), mlp_clf)
pipeline.fit(X_train, y_train)
accuracy = pipeline.score(X_valid, y_valid)
accuracy

```

1.0

▼ Triển khai MLP bằng Keras

Xây dựng bộ phân loại hình ảnh sử dụng Sequential API

Sử dụng Keras để tải tập dữ liệu

Chúng ta hãy bắt đầu bằng việc tải tập dữ liệu **Fashion MNIST**. Keras có một số hàm để tải các tập dữ liệu phổ biến trong `tf.keras.datasets`. Tập dữ liệu đã được chia sẵn cho bạn thành tập huấn luyện (training set) (60.000 hình ảnh) và tập kiểm tra (test

set) (10.000 hình ảnh), nhưng việc chia tập huấn luyện thêm để có một tập xác thực (validation set) có thể hữu ích. Chúng ta sẽ sử dụng 55.000 hình ảnh cho huấn luyện và 5.000 hình ảnh cho xác thực.

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

Tập huấn luyện chứa 60.000 hình ảnh thang độ xám (grayscale), mỗi hình có kích thước 28x28 pixel:

```
X_train.shape

(55000, 28, 28)
```

Cường độ của mỗi pixel được biểu diễn dưới dạng một byte (từ 0 đến 255):

```
X_train.dtype

dtype('uint8')
```

Chúng ta hãy **chia tỷ lệ** (scale) cường độ pixel xuống phạm vi từ 0 đến 1 và chuyển chúng thành kiểu số thực (float), bằng cách chia cho 255:

```
X_train, X_valid, X_test = X_train / 255., X_valid / 255., X_test / 255.
```

Bạn có thể vẽ một hình ảnh bằng cách sử dụng hàm `imshow()` của Matplotlib, với bản đồ màu (color map) là `'binary'`:

```
# extra code

plt.imshow(X_train[0], cmap="binary")
plt.axis('off')
plt.show()
```

Các nhãn là ID lớp (class ID) (được biểu diễn dưới dạng uint8), từ 0 đến 9:

```
y_train

array([9, 0, 0, ..., 9, 0, 2], dtype=uint8)
```

Đây là các tên lớp tương ứng:

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

Vì vậy, hình ảnh đầu tiên trong tập huấn luyện là một **ủng mắt cá chân** (ankle boot):

```
class_names[y_train[0]]
```

Chúng ta hãy xem một mẫu các hình ảnh trong tập dữ liệu:

```
# extra code - this cell generates and saves Figure 10-10

n_rows = 4
n_cols = 10
plt.figure(figsize=(n_cols * 1.2, n_rows * 1.2))
for row in range(n_rows):
    for col in range(n_cols):
        index = n_cols * row + col
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(X_train[index], cmap="binary", interpolation="nearest")
        plt.axis('off')
        plt.title(class_names[y_train[index]])
plt.subplots_adjust(wspace=0.2, hspace=0.5)

save_fig("Fashion_mnist_plot")
plt.show()
```

▼ Tạo mô hình sử dụng Sequential API

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.InputLayer(input_shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/input_layer.py:27: UserWarning: Argument `input_shape` is deprecated
warnings.warn(
```

```
# extra code - clear the session to reset the name counters
tf.keras.backend.clear_session()
tf.random.set_seed(42)

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
```

```
tf.keras.layers.Dense(10, activation="softmax")
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`inp
super().__init__(**kwargs)
```

```
model.summary()
```

```
# extra code - another way to display the model's architecture
tf.keras.utils.plot_model(model, "my_fashion_mnist_model.png", show_shapes=True)
```

```
[<Flatten name=flatten, built=True>,  
<Dense name=dense, built=True>,  
<Dense name=dense_1, built=True>,  
<Dense name=dense_2, built=True>]
```

```
hidden1 = model.layers[1]
hidden1.name
```

```
model.get_layer('dense') is hidden1
```

True

```
weights, biases = hidden1.get_weights()
weights
```

```
array([[ 1.5117422e-02, -2.3315549e-03, -3.1667426e-02, ...,
        6.5189853e-02,  3.5784915e-03,  6.5990433e-02],
       [ 1.8503718e-02,  7.3931068e-02,  2.0940952e-02, ...,
        3.3008441e-02, -1.5675846e-02,  3.8541853e-05],
       [-6.4281300e-03, -3.4955118e-02,  6.9208145e-02, ...,
        5.6569576e-02, -7.2203651e-02,  2.5110915e-02],
       ...,
       [ 6.7211270e-02, -4.3009363e-02, -3.1973369e-02, ...,
        1.6179211e-02, -5.8345214e-02,  5.2010298e-02],
       [-4.8070505e-02, -4.5199655e-02, -1.4409896e-02, ...,
        6.1760038e-02, -5.7009108e-02,  3.5401314e-02],
       [-3.3959795e-02,  2.2510223e-02,  2.3465015e-02, ...,
        -2.4398666e-02, -4.3925826e-02, -1.4634918e-02]], dtype=float32)
```

```
weights.shape
```

(784, 300)

biases

[illegible]

biases.shape

(300,)

- ✓ Biên dịch mô hình

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])
```

Điều này tương đương với:

```
# extra code - this cell is equivalent to the previous cell
model.compile(loss=tf.keras.losses.sparse_categorical_crossentropy,
```



```
optimizer=tf.keras.optimizers.SGD(),  
metrics=[tf.keras.metrics.sparse_categorical_accuracy])
```

```
# extra code - shows how to convert class ids to one-hot vectors  
tf.keras.utils.to_categorical([0, 5, 1, 0], num_classes=10)
```

```
array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
       [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],  
       [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],  
       [1., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

Lưu ý: điều quan trọng là phải đặt `num_classes` khi số lượng lớp lớn hơn ID lớp tối đa trong mẫu.

```
# extra code - shows how to convert one-hot vectors to class ids
```

```
np.argmax(  
    [[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],  
     [0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],  
     [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],  
     [1., 0., 0., 0., 0., 0., 0., 0., 0., 0.]],  
    axis=1  
)
```

```
array([0, 5, 1, 0])
```

▼ Huấn luyện và đánh giá mô hình

```
history = model.fit(X_train, y_train, epochs=30,  
                    validation_data=(X_valid, y_valid))
```

```
Epoch 1/30  
1719/1719 ————— 9s 5ms/step - loss: 1.0148 - sparse_categorical_accuracy: 0.6801 - val_loss: 0.5061 - val_sparse_categorical_accuracy: 0.6801  
Epoch 2/30  
1719/1719 ————— 8s 5ms/step - loss: 0.5098 - sparse_categorical_accuracy: 0.8251 - val_loss: 0.4490 - val_sparse_categorical_accuracy: 0.8251  
Epoch 3/30  
1719/1719 ————— 8s 5ms/step - loss: 0.4528 - sparse_categorical_accuracy: 0.8425 - val_loss: 0.4241 - val_sparse_categorical_accuracy: 0.8425  
Epoch 4/30  
1719/1719 ————— 9s 5ms/step - loss: 0.4220 - sparse_categorical_accuracy: 0.8522 - val_loss: 0.4091 - val_sparse_categorical_accuracy: 0.8522  
Epoch 5/30  
1719/1719 ————— 8s 5ms/step - loss: 0.4005 - sparse_categorical_accuracy: 0.8590 - val_loss: 0.3981 - val_sparse_categorical_accuracy: 0.8590  
Epoch 6/30  
1719/1719 ————— 9s 5ms/step - loss: 0.3838 - sparse_categorical_accuracy: 0.8650 - val_loss: 0.3902 - val_sparse_categorical_accuracy: 0.8650  
Epoch 7/30  
1719/1719 ————— 9s 5ms/step - loss: 0.3699 - sparse_categorical_accuracy: 0.8693 - val_loss: 0.3837 - val_sparse_categorical_accuracy: 0.8693  
Epoch 8/30  
1719/1719 ————— 8s 4ms/step - loss: 0.3579 - sparse_categorical_accuracy: 0.8722 - val_loss: 0.3770 - val_sparse_categorical_accuracy: 0.8722  
Epoch 9/30  
1719/1719 ————— 9s 5ms/step - loss: 0.3472 - sparse_categorical_accuracy: 0.8762 - val_loss: 0.3722 - val_sparse_categorical_accuracy: 0.8762  
Epoch 10/30  
1719/1719 ————— 8s 5ms/step - loss: 0.3375 - sparse_categorical_accuracy: 0.8789 - val_loss: 0.3679 - val_sparse_categorical_accuracy: 0.8789  
Epoch 11/30  
1719/1719 ————— 7s 4ms/step - loss: 0.3287 - sparse_categorical_accuracy: 0.8823 - val_loss: 0.3636 - val_sparse_categorical_accuracy: 0.8823  
Epoch 12/30  
1719/1719 ————— 9s 5ms/step - loss: 0.3203 - sparse_categorical_accuracy: 0.8842 - val_loss: 0.3604 - val_sparse_categorical_accuracy: 0.8842  
Epoch 13/30  
1719/1719 ————— 8s 5ms/step - loss: 0.3126 - sparse_categorical_accuracy: 0.8863 - val_loss: 0.3547 - val_sparse_categorical_accuracy: 0.8863  
Epoch 14/30  
1719/1719 ————— 10s 5ms/step - loss: 0.3052 - sparse_categorical_accuracy: 0.8897 - val_loss: 0.3531 - val_sparse_categorical_accuracy: 0.8897  
Epoch 15/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2985 - sparse_categorical_accuracy: 0.8925 - val_loss: 0.3505 - val_sparse_categorical_accuracy: 0.8925  
Epoch 16/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2922 - sparse_categorical_accuracy: 0.8944 - val_loss: 0.3505 - val_sparse_categorical_accuracy: 0.8944  
Epoch 17/30  
1719/1719 ————— 8s 4ms/step - loss: 0.2863 - sparse_categorical_accuracy: 0.8971 - val_loss: 0.3482 - val_sparse_categorical_accuracy: 0.8971  
Epoch 18/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2806 - sparse_categorical_accuracy: 0.8990 - val_loss: 0.3444 - val_sparse_categorical_accuracy: 0.8990  
Epoch 19/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2749 - sparse_categorical_accuracy: 0.9012 - val_loss: 0.3435 - val_sparse_categorical_accuracy: 0.9012  
Epoch 20/30  
1719/1719 ————— 8s 5ms/step - loss: 0.2696 - sparse_categorical_accuracy: 0.9027 - val_loss: 0.3446 - val_sparse_categorical_accuracy: 0.9027  
Epoch 21/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2646 - sparse_categorical_accuracy: 0.9044 - val_loss: 0.3429 - val_sparse_categorical_accuracy: 0.9044  
Epoch 22/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2596 - sparse_categorical_accuracy: 0.9065 - val_loss: 0.3421 - val_sparse_categorical_accuracy: 0.9065  
Epoch 23/30  
1719/1719 ————— 8s 4ms/step - loss: 0.2550 - sparse_categorical_accuracy: 0.9073 - val_loss: 0.3425 - val_sparse_categorical_accuracy: 0.9073  
Epoch 24/30  
1719/1719 ————— 9s 5ms/step - loss: 0.2505 - sparse_categorical_accuracy: 0.9095 - val_loss: 0.3418 - val_sparse_categorical_accuracy: 0.9095  
Epoch 25/30
```

```
1719/1719 ————— 9s 5ms/step - loss: 0.2460 - sparse_categorical_accuracy: 0.9113 - val_loss: 0.3403 - val_sparse_categorical_accuracy: 0.9113
Epoch 26/30
1719/1719 ————— 8s 5ms/step - loss: 0.2418 - sparse_categorical_accuracy: 0.9130 - val_loss: 0.3395 - val_sparse_categorical_accuracy: 0.9130
Epoch 27/30
1719/1719 ————— 9s 5ms/step - loss: 0.2377 - sparse_categorical_accuracy: 0.9141 - val_loss: 0.3394 - val_sparse_categorical_accuracy: 0.9141
Epoch 28/30
1719/1719 ————— 10s 5ms/step - loss: 0.2337 - sparse_categorical_accuracy: 0.9158 - val_loss: 0.3395 - val_sparse_categorical_accuracy: 0.9158
Epoch 29/30
```

```
history.params
```

```
{'verbose': 'auto', 'epochs': 30, 'steps': 1719}
```

```
print(history.epoch)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]
```

```
import matplotlib.pyplot as plt
import pandas as pd

pd.DataFrame(history.history).plot(
    figsize=(8, 5), xlim=[0, 29], ylim=[0, 1], grid=True, xlabel="Epoch",
    style=["r--", "r--.", "b-", "b-*"])
plt.legend(loc="lower left") # extra code
save_fig("keras_learning_curves_plot") # extra code
plt.show()
```

```
# extra code - shows how to shift the training curve by -1/2 epoch
plt.figure(figsize=(8, 5))
for key, style in zip(history.history, ["r--", "r--.", "b-", "b-*"]):
    epochs = np.array(history.epoch) + (0 if key.startswith("val_") else -0.5)
    plt.plot(epochs, history.history[key], style, label=key)
plt.xlabel("Epoch")
plt.axis([-0.5, 29, 0., 1])
plt.legend(loc="lower left")
plt.grid()
plt.show()
```

```
model.evaluate(X_test, y_test)
```

```
313/313 ————— 1s 3ms/step - loss: 0.3539 - sparse_categorical_accuracy: 0.8776  
[0.3554833233356476, 0.876200203132629]
```

✓ Sử dụng mô hình để đưa ra dự đoán

```
X_new = X_test[:3]  
y_proba = model.predict(X_new)  
y_proba.round(2)
```

```
WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_dist  
1/1 ————— 0s 99ms/step  
array([[0. , 0. , 0. , 0. , 0. , 0.11, 0. , 0.01, 0. , 0.88],  
       [0. , 0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ],  
       [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]],  
      dtype=float32)
```

```
y_pred = y_proba.argmax(axis=-1)  
y_pred
```

```
array([9, 2, 1])
```

```
np.array(class_names)[y_pred]
```

```
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11'))
```

```
y_new = y_test[:3]  
y_new
```

```
array([9, 2, 1], dtype=uint8)
```

```
# extra code - this cell generates and saves Figure 10-12  
plt.figure(figsize=(7.2, 2.4))  
for index, image in enumerate(X_new):  
    plt.subplot(1, 3, index + 1)  
    plt.imshow(image, cmap="binary", interpolation="nearest")  
    plt.axis('off')  
    plt.title(class_names[y_test[index]])  
plt.subplots_adjust(wspace=0.2, hspace=0.5)  
save_fig('fashion_mnist_images_plot', tight_layout=False)  
plt.show()
```

✓ Xây dựng MLP Hồi quy sử dụng Sequential API

Chúng ta hãy tải, chia tách và chia tỷ lệ tập dữ liệu nhà ở California (bản gốc, không phải bản đã được chỉnh sửa như trong chương 2):

```
# extra code - load and split the California housing dataset, like earlier
housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)
```

```
tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization(input_shape=X_train.shape[1:])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

```
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/preprocessing/tf_data_layer.py:19: UserWarning: Do not pass an `input_shape` argument to the `Normalization` layer.
super().__init__(**kwargs)
363/363 ————— 4s 6ms/step - RootMeanSquaredError: 1.2733 - loss: 1.7501 - val_RootMeanSquaredError: 0.9047 - val_
Epoch 2/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6422 - loss: 0.4128 - val_RootMeanSquaredError: 0.7913 - val_
Epoch 3/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6084 - loss: 0.3704 - val_RootMeanSquaredError: 1.3052 - val_
Epoch 4/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5910 - loss: 0.3495 - val_RootMeanSquaredError: 1.4098 - val_
Epoch 5/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5781 - loss: 0.3343 - val_RootMeanSquaredError: 1.0848 - val_
Epoch 6/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5691 - loss: 0.3240 - val_RootMeanSquaredError: 1.2566 - val_
Epoch 7/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5603 - loss: 0.3140 - val_RootMeanSquaredError: 0.7583 - val_
Epoch 8/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5524 - loss: 0.3053 - val_RootMeanSquaredError: 1.3315 - val_
Epoch 9/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5489 - loss: 0.3013 - val_RootMeanSquaredError: 1.0135 - val_
Epoch 10/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5477 - loss: 0.3001 - val_RootMeanSquaredError: 1.2863 - val_
Epoch 11/20
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.5455 - loss: 0.2976 - val_RootMeanSquaredError: 0.7738 - val_
Epoch 12/20
363/363 ————— 2s 5ms/step - RootMeanSquaredError: 0.5392 - loss: 0.2909 - val_RootMeanSquaredError: 1.1170 - val_
Epoch 13/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5357 - loss: 0.2871 - val_RootMeanSquaredError: 0.6581 - val_
Epoch 14/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5334 - loss: 0.2846 - val_RootMeanSquaredError: 0.8507 - val_
Epoch 15/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5315 - loss: 0.2826 - val_RootMeanSquaredError: 0.5834 - val_
Epoch 16/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5271 - loss: 0.2779 - val_RootMeanSquaredError: 0.7502 - val_
Epoch 17/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5228 - loss: 0.2734 - val_RootMeanSquaredError: 0.5567 - val_
Epoch 18/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5220 - loss: 0.2726 - val_RootMeanSquaredError: 1.0042 - val_
Epoch 19/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5196 - loss: 0.2701 - val_RootMeanSquaredError: 0.5902 - val_
Epoch 20/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5191 - loss: 0.2696 - val_RootMeanSquaredError: 0.9602 - val_
162/162 ————— 0s 2ms/step - RootMeanSquaredError: 0.5346 - loss: 0.2860
WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_dist
1/1 ————— 0s 83ms/step
```

rmse_test

0.5394777059555054

```
y_pred
```

```
array([[0.41626406],  
       [1.286773  ],  
       [5.0948157  ]], dtype=float32)
```

✧ Xây dựng các mô hình phức tạp sử dụng Functional API

Không phải tất cả các mô hình mạng nơ-ron đều là **tuần tự** (sequential) đơn thuần. Một số có thể có cấu trúc **phức tạp** (complex topologies). Một số có thể có **nhiều đầu vào** (multiple inputs) và/hoặc **nhiều đầu ra** (multiple outputs). Ví dụ, một mạng nơ-ron Wide & Deep (xem [bài báo](#)) kết nối tất cả hoặc một phần của các đầu vào **trực tiếp** với lớp đầu ra.

```
# extra code - reset the name counters and make the code reproducible  
tf.keras.backend.clear_session()  
tf.random.set_seed(42)
```

```
normalization_layer = tf.keras.layers.Normalization()  
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")  
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")  
concat_layer = tf.keras.layers.Concatenate()  
output_layer = tf.keras.layers.Dense(1)  
  
input_ = tf.keras.layers.Input(shape=X_train.shape[1:])  
normalized = normalization_layer(input_)  
hidden1 = hidden_layer1(normalized)  
hidden2 = hidden_layer2(hidden1)  
concat = concat_layer([normalized, hidden2])  
output = output_layer(concat)  
  
model = tf.keras.Model(inputs=[input_], outputs=[output])
```

```
model.summary()
```

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)  
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])  
normalization_layer.adapt(X_train)  
history = model.fit(X_train, y_train, epochs=20,  
                    validation_data=(X_valid, y_valid))  
mse_test = model.evaluate(X_test, y_test)  
y_pred = model.predict(X_new)
```

```
Epoch 1/20  
363/363 ————— 3s 4ms/step - RootMeanSquaredError: 1.5242 - loss: 2.4598 - val_RootMeanSquaredError: 0.8349 - val_  
Epoch 2/20  
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.7367 - loss: 0.5446 - val_RootMeanSquaredError: 0.8662 - val_  
Epoch 3/20  
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6549 - loss: 0.4294 - val_RootMeanSquaredError: 0.8776 - val_  
Epoch 4/20  
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6241 - loss: 0.3899 - val_RootMeanSquaredError: 0.6805 - val_  
Epoch 5/20  
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6095 - loss: 0.3718 - val_RootMeanSquaredError: 1.1873 - val_  
Epoch 6/20  
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6011 - loss: 0.3616 - val_RootMeanSquaredError: 1.2699 - val_
```

```

Epoch 7/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5950 - loss: 0.3542 - val_RootMeanSquaredError: 1.4385 - val_
Epoch 8/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5909 - loss: 0.3493 - val_RootMeanSquaredError: 1.4064 - val_
Epoch 9/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5860 - loss: 0.3435 - val_RootMeanSquaredError: 1.2093 - val_
Epoch 10/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5806 - loss: 0.3373 - val_RootMeanSquaredError: 1.2108 - val_
Epoch 11/20
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.5769 - loss: 0.3329 - val_RootMeanSquaredError: 1.1946 - val_
Epoch 12/20
363/363 ————— 2s 3ms/step - RootMeanSquaredError: 0.5757 - loss: 0.3316 - val_RootMeanSquaredError: 1.1240 - val_
Epoch 13/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5713 - loss: 0.3265 - val_RootMeanSquaredError: 1.0315 - val_
Epoch 14/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5687 - loss: 0.3235 - val_RootMeanSquaredError: 0.9980 - val_
Epoch 15/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5660 - loss: 0.3205 - val_RootMeanSquaredError: 0.9636 - val_
Epoch 16/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5659 - loss: 0.3203 - val_RootMeanSquaredError: 1.0804 - val_
Epoch 17/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5630 - loss: 0.3171 - val_RootMeanSquaredError: 1.1354 - val_
Epoch 18/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5623 - loss: 0.3163 - val_RootMeanSquaredError: 1.1551 - val_
Epoch 19/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5605 - loss: 0.3142 - val_RootMeanSquaredError: 1.0353 - val_
Epoch 20/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5596 - loss: 0.3132 - val_RootMeanSquaredError: 1.0144 - val_
162/162 ————— 0s 2ms/step - RootMeanSquaredError: 0.5547 - loss: 0.3078
1/1 ————— 0s 123ms/step

```

Điều gì sẽ xảy ra nếu bạn muốn gửi các **tập con khác nhau** của các đặc trưng đầu vào (input features) thông qua các đường **rộng (wide)** hoặc **sâu (deep)**? Chúng ta sẽ gửi 5 đặc trưng (đặc trưng 0 đến 4) qua đường rộng, và 6 đặc trưng qua đường sâu (đặc trưng 2 đến 7). Lưu ý rằng 3 đặc trưng sẽ đi qua cả hai (đặc trưng 2, 3 và 4).

```
tf.random.set_seed(42) # extra code
```

```

input_wide = tf.keras.layers.Input(shape=[5]) # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6]) # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])

```

```

optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])

```

```

X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

```

```

norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit((X_train_wide, X_train_deep), y_train, epochs=20,
                    validation_data=((X_valid_wide, X_valid_deep), y_valid))
mse_test = model.evaluate((X_test_wide, X_test_deep), y_test)
y_pred = model.predict((X_new_wide, X_new_deep))

```

```

Epoch 1/20
363/363 ————— 3s 4ms/step - RootMeanSquaredError: 1.8544 - loss: 3.5817 - val_RootMeanSquaredError: 1.6045 - val_
Epoch 2/20
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.8572 - loss: 0.7362 - val_RootMeanSquaredError: 0.9409 - val_
Epoch 3/20
363/363 ————— 2s 5ms/step - RootMeanSquaredError: 0.7309 - loss: 0.5344 - val_RootMeanSquaredError: 0.7233 - val_
Epoch 4/20
363/363 ————— 2s 3ms/step - RootMeanSquaredError: 0.6778 - loss: 0.4597 - val_RootMeanSquaredError: 0.6762 - val_
Epoch 5/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6460 - loss: 0.4174 - val_RootMeanSquaredError: 0.6068 - val_
Epoch 6/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6283 - loss: 0.3949 - val_RootMeanSquaredError: 0.6472 - val_
Epoch 7/20

```

```

363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.6163 - loss: 0.3800 - val_RootMeanSquaredError: 0.5849 - val_
Epoch 8/20
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.6088 - loss: 0.3708 - val_RootMeanSquaredError: 0.5979 - val_
Epoch 9/20
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.6031 - loss: 0.3639 - val_RootMeanSquaredError: 0.6484 - val_
Epoch 10/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5994 - loss: 0.3595 - val_RootMeanSquaredError: 0.7620 - val_
Epoch 11/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5946 - loss: 0.3537 - val_RootMeanSquaredError: 1.0205 - val_
Epoch 12/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5933 - loss: 0.3521 - val_RootMeanSquaredError: 1.1222 - val_
Epoch 13/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5909 - loss: 0.3493 - val_RootMeanSquaredError: 1.2141 - val_
Epoch 14/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5894 - loss: 0.3476 - val_RootMeanSquaredError: 1.1472 - val_
Epoch 15/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5873 - loss: 0.3450 - val_RootMeanSquaredError: 1.1560 - val_
Epoch 16/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5858 - loss: 0.3433 - val_RootMeanSquaredError: 1.0008 - val_
Epoch 17/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5836 - loss: 0.3406 - val_RootMeanSquaredError: 0.7371 - val_
Epoch 18/20
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5794 - loss: 0.3358 - val_RootMeanSquaredError: 0.7760 - val_
Epoch 19/20
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.5763 - loss: 0.3322 - val_RootMeanSquaredError: 0.7288 - val_
Epoch 20/20
363/363 ————— 2s 5ms/step - RootMeanSquaredError: 0.5747 - loss: 0.3304 - val_RootMeanSquaredError: 0.9795 - val_
162/162 ————— 0s 2ms/step - RootMeanSquaredError: 0.5837 - loss: 0.3408
1/1 ————— 0s 89ms/step

```

Thêm một **đầu ra phụ** (auxiliary output) cho **chuẩn hóa** (regularization):

```

tf.keras.backend.clear_session()
tf.random.set_seed(42)

```

```

input_wide = tf.keras.layers.Input(shape=[5]) # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6]) # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
aux_output = tf.keras.layers.Dense(1)(hidden2)
model = tf.keras.Model(inputs=[input_wide, input_deep],
                        outputs=[output, aux_output])

```

```

optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss=["mse", "mse"], loss_weights=(0.9, 0.1), optimizer=optimizer,
              metrics=["RootMeanSquaredError"])

```

```

# Chuẩn hóa dữ liệu
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)

# Compile mô hình với 2 loss và 2 metrics (1 cho mỗi đầu ra)
model.compile(
    optimizer="adam",
    loss=["mse", "mse"],
    metrics=[["RootMeanSquaredError"], ["RootMeanSquaredError"]]
)

# Huấn luyện mô hình
history = model.fit(
    (X_train_wide, X_train_deep),
    (y_train, y_train), # 2 nhãn cho 2 đầu ra
    epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid))
)

```

```

Epoch 1/20
363/363 ————— 3s 4ms/step - dense_2_RootMeanSquaredError: 1.7547 - dense_2_loss: 3.1860 - dense_3_RootMeanSquared
Epoch 2/20

```

```

363/363 ————— 1s 4ms/step - dense_2_RootMeanSquaredError: 0.8075 - dense_2_loss: 0.6529 - dense_3_RootMeanSquared
Epoch 3/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.7185 - dense_2_loss: 0.5165 - dense_3_RootMeanSquared
Epoch 4/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6810 - dense_2_loss: 0.4640 - dense_3_RootMeanSquared
Epoch 5/20
363/363 ————— 1s 4ms/step - dense_2_RootMeanSquaredError: 0.6570 - dense_2_loss: 0.4318 - dense_3_RootMeanSquared
Epoch 6/20
363/363 ————— 2s 6ms/step - dense_2_RootMeanSquaredError: 0.6406 - dense_2_loss: 0.4105 - dense_3_RootMeanSquared
Epoch 7/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6288 - dense_2_loss: 0.3955 - dense_3_RootMeanSquared
Epoch 8/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6208 - dense_2_loss: 0.3855 - dense_3_RootMeanSquared
Epoch 9/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6147 - dense_2_loss: 0.3781 - dense_3_RootMeanSquared
Epoch 10/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6095 - dense_2_loss: 0.3716 - dense_3_RootMeanSquared
Epoch 11/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6059 - dense_2_loss: 0.3672 - dense_3_RootMeanSquared
Epoch 12/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.6013 - dense_2_loss: 0.3617 - dense_3_RootMeanSquared
Epoch 13/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5984 - dense_2_loss: 0.3583 - dense_3_RootMeanSquared
Epoch 14/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5951 - dense_2_loss: 0.3543 - dense_3_RootMeanSquared
Epoch 15/20
363/363 ————— 3s 4ms/step - dense_2_RootMeanSquaredError: 0.5924 - dense_2_loss: 0.3510 - dense_3_RootMeanSquared
Epoch 16/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5886 - dense_2_loss: 0.3466 - dense_3_RootMeanSquared
Epoch 17/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5872 - dense_2_loss: 0.3449 - dense_3_RootMeanSquared
Epoch 18/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5836 - dense_2_loss: 0.3407 - dense_3_RootMeanSquared
Epoch 19/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5823 - dense_2_loss: 0.3392 - dense_3_RootMeanSquared
Epoch 20/20
363/363 ————— 1s 3ms/step - dense_2_RootMeanSquaredError: 0.5795 - dense_2_loss: 0.3360 - dense_3_RootMeanSquared

```

```

eval_results = model.evaluate((X_test_wide, X_test_deep), (y_test, y_test))
weighted_sum_of_losses, main_loss, aux_loss, main_rmse, aux_rmse = eval_results

```

```

162/162 ————— 0s 2ms/step - dense_2_RootMeanSquaredError: 0.5775 - dense_2_loss: 0.3336 - dense_3_RootMeanSquared

```

```

y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))

```

```

1/1 ————— 0s 105ms/step

```

```

y_pred_tuple = model.predict((X_new_wide, X_new_deep))
y_pred = dict(zip(model.output_names, y_pred_tuple))

```

```

1/1 ————— 0s 40ms/step

```

✧ Sử dụng Subclassing API để Xây dựng các Mô hình Động

```

class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs) # needed to support naming the model
        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)
        self.main_output = tf.keras.layers.Dense(1)
        self.aux_output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])
        output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return output, aux_output

```



```
tf.random.set_seed(42) # extra code - just for reproducibility
model = WideAndDeepModel(30, activation="relu", name="my_cool_model")
```

```
import tensorflow as tf

# --- 1. Compile mô hình ---
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(
    loss=["mse", "mse"], # 2 losses
    loss_weights=[0.9, 0.1], # khớp với 2 outputs
    optimizer=optimizer,
    metrics=[["RootMeanSquaredError"], ["RootMeanSquaredError"]]
)

# --- 2. Chuẩn hóa dữ liệu ---
model.norm_layer_wide.adapt(X_train_wide)
model.norm_layer_deep.adapt(X_train_deep)

# --- 3. Huấn luyện mô hình ---
history = model.fit(
    (X_train_wide, X_train_deep),
    (y_train, y_train), # 2 outputs tương ứng
    epochs=10,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid))
)

# --- 4. Đánh giá mô hình ---
eval_results = model.evaluate(
    (X_test_wide, X_test_deep),
    (y_test, y_test)
)

# --- 5. Kiểm tra số phần tử trả về ---
print("Metric names:", model.metrics_names)
print("Eval results:", eval_results)

# 📌 Trong kiến trúc Wide & Deep chuẩn, eval_results thường có 4 phần tử:
# ['loss', 'output_1_loss', 'root_mean_squared_error', 'root_mean_squared_error_1']

total_loss = eval_results[0]
main_loss = eval_results[1]
main_rmse = eval_results[2]
aux_rmse = eval_results[3]

print(f"Total loss: {total_loss:.4f}")
print(f"Main loss: {main_loss:.4f}")
print(f"Main RMSE: {main_rmse:.4f}")
print(f"Aux RMSE: {aux_rmse:.4f}")

# --- 6. Dự đoán ---
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

```
Epoch 1/10
363/363 ————— 4s 7ms/step - RootMeanSquaredError: 0.5976 - RootMeanSquaredError_1: 0.6584 - loss: 0.3648 - mse_lo
Epoch 2/10
363/363 ————— 1s 4ms/step - RootMeanSquaredError: 0.5898 - RootMeanSquaredError_1: 0.6508 - loss: 0.3555 - mse_lo
Epoch 3/10
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5871 - RootMeanSquaredError_1: 0.6449 - loss: 0.3519 - mse_lo
Epoch 4/10
363/363 ————— 1s 4ms/step - RootMeanSquaredError: 0.5850 - RootMeanSquaredError_1: 0.6411 - loss: 0.3492 - mse_lo
Epoch 5/10
363/363 ————— 1s 4ms/step - RootMeanSquaredError: 0.5840 - RootMeanSquaredError_1: 0.6378 - loss: 0.3476 - mse_lo
Epoch 6/10
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5826 - RootMeanSquaredError_1: 0.6348 - loss: 0.3459 - mse_lo
Epoch 7/10
363/363 ————— 1s 3ms/step - RootMeanSquaredError: 0.5807 - RootMeanSquaredError_1: 0.6312 - loss: 0.3434 - mse_lo
Epoch 8/10
363/363 ————— 2s 4ms/step - RootMeanSquaredError: 0.5780 - RootMeanSquaredError_1: 0.6294 - loss: 0.3403 - mse_lo
Epoch 9/10
363/363 ————— 2s 6ms/step - RootMeanSquaredError: 0.5753 - RootMeanSquaredError_1: 0.6259 - loss: 0.3371 - mse_lo
Epoch 10/10
363/363 ————— 2s 5ms/step - RootMeanSquaredError: 0.5736 - RootMeanSquaredError_1: 0.6254 - loss: 0.3353 - mse_lo
162/162 ————— 0s 2ms/step - RootMeanSquaredError: 0.5761 - RootMeanSquaredError_1: 0.6302 - loss: 0.3385 - mse_lo
Metric names: ['loss', 'compile_metrics', 'mse_loss', 'mse_loss']
Eval results: [0.33299073576927185, 0.5714923739433289, 0.5714923739433289, 0.390562504529953, 0.390562504529953, 0.624880373477
Total loss: 0.3330
Main loss: 0.5715
Main RMSE: 0.5715
```

✓ Lưu và Phục hồi Mô hình

```
# extra code - delete the directory, in case it already exists
```

```
import shutil
```

```
shutil.rmtree("my_keras_model", ignore_errors=True)
```

```
from keras.utils import custom_object_scope
```

```
with custom_object_scope({'WideAndDeepModel': WideAndDeepModel}):  
    model = tf.keras.models.load_model("my_keras_model.keras")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802: UserWarning: Skipping variable loading for optimizer  
saveable.load_own_variables(weights_store.get(inner_path))
```

```
# extra code - show the contents of the my_keras_model/ directory  
for path in sorted(Path("my_keras_model").glob("**/*")):  
    print(path)
```

```
import tensorflow as tf  
import shutil  
import numpy as np
```

```
# 🍷 1. Khai báo lớp mô hình tùy chỉnh
```

```
@tf.keras.utils.register_keras_serializable()
```

```
class WideAndDeepModel(tf.keras.Model):
```

```
    def __init__(self, units=30, activation="relu", **kwargs):  
        super().__init__(**kwargs)  
        self.norm_layer_wide = tf.keras.layers.Normalization()  
        self.norm_layer_deep = tf.keras.layers.Normalization()  
        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)  
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)  
        self.main_output = tf.keras.layers.Dense(1)  
        self.aux_output = tf.keras.layers.Dense(1)
```

```
    def call(self, inputs):  
        input_wide, input_deep = inputs  
        norm_wide = self.norm_layer_wide(input_wide)  
        norm_deep = self.norm_layer_deep(input_deep)  
        hidden1 = self.hidden1(norm_deep)  
        hidden2 = self.hidden2(hidden1)  
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])  
        output = self.main_output(concat)  
        aux_output = self.aux_output(hidden2)  
        return output, aux_output
```

```
# 📊 2. Giả lập dữ liệu (bạn thay bằng dữ liệu thật)
```

```
X_train_wide = np.random.rand(100, 5).astype("float32")
```

```
X_train_deep = np.random.rand(100, 10).astype("float32")
```

```
y_train = np.random.rand(100, 1).astype("float32")
```

```
X_new_wide = np.random.rand(3, 5).astype("float32")
```

```
X_new_deep = np.random.rand(3, 10).astype("float32")
```

```
# 🛠 3. Tạo model
```

```
model = WideAndDeepModel(30, "relu")
```

```
# ⚡ 4. Adapt normalization layers trước khi build
```

```
model.norm_layer_wide.adapt(X_train_wide)
```

```
model.norm_layer_deep.adapt(X_train_deep)
```

```
# 🏗 5. Build model bằng cách gọi 1 lần
```

```
model((X_train_wide[:1], X_train_deep[:1]))
```

```
# 🛠️ 6. Compile
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss=["mse", "mse"],
    loss_weights=[0.9, 0.1],
    metrics=[["RootMeanSquaredError"], ["RootMeanSquaredError"]]
)

# (Tuỳ chọn) train model một chút
model.fit((X_train_wide, X_train_deep), (y_train, y_train), epochs=1, verbose=0)

# 🧹 7. Xóa file cũ nếu có
shutil.rmtree("my_keras_model.keras", ignore_errors=True)

# 💾 8. Lưu mô hình (bây giờ đã có state)
model.save("my_keras_model.keras")

# 📄 9. Load mô hình
model = tf.keras.models.load_model("my_keras_model.keras")

# ✅ 10. Predict thành công
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
print("Main prediction shape:", y_pred_main.shape)
print("Aux prediction shape:", y_pred_aux.shape)
```

```
1/1 ————— 0s 99ms/step
Main prediction shape: (3, 1)
Aux prediction shape: (3, 1)
```

```
# ❌ Sai:
# model.save_weights("my_weights")

# ✅ Đúng:
model.save_weights("my_weights.weights.h5")
```

```
model.load_weights("my_weights.weights.h5")
```

```
# extra code - show the list of my_weights.* files
for path in sorted(Path().glob("my_weights.*")):
    print(path)
```

```
my_weights.weights.h5
```

▼ Sử dụng Callbacks

```
shutil.rmtree("my_checkpoints", ignore_errors=True) # extra code
```

```
import tensorflow as tf
import numpy as np

# 🧪 1. Giả lập dữ liệu đúng shape
# Giả sử wide = 5 chiều, deep = 6 chiều
X_train_wide = np.random.rand(100, 5).astype("float32")
X_train_deep = np.random.rand(100, 6).astype("float32")
y_train = np.random.rand(100, 1).astype("float32")

X_valid_wide = np.random.rand(20, 5).astype("float32")
X_valid_deep = np.random.rand(20, 6).astype("float32")
y_valid = np.random.rand(20, 1).astype("float32")

# 🧠 2. Định nghĩa model
@tf.keras.utils.register_keras_serializable()
class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
```

```

        self.hidden1 = tf.keras.layers.Dense(units, activation=activation)
        self.hidden2 = tf.keras.layers.Dense(units, activation=activation)
        self.main_output = tf.keras.layers.Dense(1)
        self.aux_output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide, hidden2])
        output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return output, aux_output

# 🛠️ 3. Tạo model
model = WideAndDeepModel(30, "relu")

# ⚡ 4. Adapt normalization layer theo đúng shape dữ liệu
model.norm_layer_wide.adapt(X_train_wide)
model.norm_layer_deep.adapt(X_train_deep)

# 🏠 5. Build model trước khi train
model((X_train_wide[:1], X_train_deep[:1]))

# ⚙️ 6. Compile
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss=["mse", "mse"],
    loss_weights=[0.9, 0.1],
    metrics=[["RootMeanSquaredError"], ["RootMeanSquaredError"]]
)

# 📁 7. Callback lưu trọng số theo từng epoch (đúng định dạng Keras 3)
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint(
    "my_checkpoints_{epoch:02d}.weights.h5", # ✅ đúng đuôi
    save_weights_only=True
)

# 🏃 8. Train model
history = model.fit(
    (X_train_wide, X_train_deep), (y_train, y_train),
    epochs=10,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid)),
    callbacks=[checkpoint_cb]
)

print("✅ Training hoàn tất, checkpoint đã lưu!")

```

```

Epoch 1/10
4/4 ————— 3s 480ms/step - RootMeanSquaredError: 0.8829 - RootMeanSquaredError_1: 0.8132 - loss: 0.7680 - mse_loss:
Epoch 2/10
4/4 ————— 0s 125ms/step - RootMeanSquaredError: 0.7985 - RootMeanSquaredError_1: 0.7756 - loss: 0.6346 - mse_loss:
Epoch 3/10
4/4 ————— 0s 73ms/step - RootMeanSquaredError: 0.7373 - RootMeanSquaredError_1: 0.7391 - loss: 0.5444 - mse_loss:
Epoch 4/10
4/4 ————— 0s 73ms/step - RootMeanSquaredError: 0.6932 - RootMeanSquaredError_1: 0.7020 - loss: 0.4823 - mse_loss:
Epoch 5/10
4/4 ————— 0s 70ms/step - RootMeanSquaredError: 0.6647 - RootMeanSquaredError_1: 0.6643 - loss: 0.4423 - mse_loss:
Epoch 6/10
4/4 ————— 0s 55ms/step - RootMeanSquaredError: 0.6485 - RootMeanSquaredError_1: 0.6266 - loss: 0.4181 - mse_loss:
Epoch 7/10
4/4 ————— 0s 43ms/step - RootMeanSquaredError: 0.6398 - RootMeanSquaredError_1: 0.5888 - loss: 0.4034 - mse_loss:
Epoch 8/10
4/4 ————— 0s 40ms/step - RootMeanSquaredError: 0.6335 - RootMeanSquaredError_1: 0.5519 - loss: 0.3920 - mse_loss:
Epoch 9/10
4/4 ————— 0s 40ms/step - RootMeanSquaredError: 0.6267 - RootMeanSquaredError_1: 0.5168 - loss: 0.3804 - mse_loss:
Epoch 10/10
4/4 ————— 0s 43ms/step - RootMeanSquaredError: 0.6177 - RootMeanSquaredError_1: 0.4842 - loss: 0.3671 - mse_loss:
✅ Training hoàn tất, checkpoint đã lưu!

```

```

early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,
                                                    restore_best_weights=True)

history = model.fit(
    (X_train_wide, X_train_deep), (y_train, y_train), epochs=100,

```

```
validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid)),
callbacks=[checkpoint_cb, early_stopping_cb])
```

```
Epoch 1/100
4/4 ————— 0s 66ms/step - RootMeanSquaredError: 0.6065 - RootMeanSquaredError_1: 0.4559 - loss: 0.3520 - mse_loss
Epoch 2/100
4/4 ————— 0s 44ms/step - RootMeanSquaredError: 0.5938 - RootMeanSquaredError_1: 0.4319 - loss: 0.3362 - mse_loss
Epoch 3/100
4/4 ————— 0s 46ms/step - RootMeanSquaredError: 0.5805 - RootMeanSquaredError_1: 0.4122 - loss: 0.3205 - mse_loss
Epoch 4/100
4/4 ————— 0s 42ms/step - RootMeanSquaredError: 0.5677 - RootMeanSquaredError_1: 0.3968 - loss: 0.3060 - mse_loss
Epoch 5/100
4/4 ————— 0s 42ms/step - RootMeanSquaredError: 0.5555 - RootMeanSquaredError_1: 0.3851 - loss: 0.2928 - mse_loss
Epoch 6/100
4/4 ————— 0s 43ms/step - RootMeanSquaredError: 0.5445 - RootMeanSquaredError_1: 0.3761 - loss: 0.2812 - mse_loss
Epoch 7/100
4/4 ————— 0s 47ms/step - RootMeanSquaredError: 0.5346 - RootMeanSquaredError_1: 0.3692 - loss: 0.2712 - mse_loss
Epoch 8/100
4/4 ————— 0s 45ms/step - RootMeanSquaredError: 0.5256 - RootMeanSquaredError_1: 0.3638 - loss: 0.2622 - mse_loss
Epoch 9/100
4/4 ————— 0s 52ms/step - RootMeanSquaredError: 0.5172 - RootMeanSquaredError_1: 0.3593 - loss: 0.2540 - mse_loss
Epoch 10/100
4/4 ————— 0s 49ms/step - RootMeanSquaredError: 0.5092 - RootMeanSquaredError_1: 0.3554 - loss: 0.2463 - mse_loss
Epoch 11/100
4/4 ————— 0s 47ms/step - RootMeanSquaredError: 0.5016 - RootMeanSquaredError_1: 0.3518 - loss: 0.2391 - mse_loss
Epoch 12/100
4/4 ————— 0s 47ms/step - RootMeanSquaredError: 0.4943 - RootMeanSquaredError_1: 0.3482 - loss: 0.2324 - mse_loss
Epoch 13/100
4/4 ————— 0s 41ms/step - RootMeanSquaredError: 0.4872 - RootMeanSquaredError_1: 0.3447 - loss: 0.2258 - mse_loss
Epoch 14/100
4/4 ————— 0s 44ms/step - RootMeanSquaredError: 0.4802 - RootMeanSquaredError_1: 0.3412 - loss: 0.2194 - mse_loss
Epoch 15/100
4/4 ————— 0s 44ms/step - RootMeanSquaredError: 0.4732 - RootMeanSquaredError_1: 0.3378 - loss: 0.2132 - mse_loss
Epoch 16/100
4/4 ————— 0s 45ms/step - RootMeanSquaredError: 0.4663 - RootMeanSquaredError_1: 0.3344 - loss: 0.2072 - mse_loss
Epoch 17/100
4/4 ————— 0s 42ms/step - RootMeanSquaredError: 0.4595 - RootMeanSquaredError_1: 0.3312 - loss: 0.2013 - mse_loss
Epoch 18/100
4/4 ————— 0s 47ms/step - RootMeanSquaredError: 0.4529 - RootMeanSquaredError_1: 0.3282 - loss: 0.1957 - mse_loss
Epoch 19/100
4/4 ————— 0s 43ms/step - RootMeanSquaredError: 0.4465 - RootMeanSquaredError_1: 0.3252 - loss: 0.1903 - mse_loss
Epoch 20/100
4/4 ————— 0s 45ms/step - RootMeanSquaredError: 0.4402 - RootMeanSquaredError_1: 0.3223 - loss: 0.1851 - mse_loss
Epoch 21/100
4/4 ————— 0s 50ms/step - RootMeanSquaredError: 0.4342 - RootMeanSquaredError_1: 0.3195 - loss: 0.1802 - mse_loss
Epoch 22/100
4/4 ————— 0s 47ms/step - RootMeanSquaredError: 0.4283 - RootMeanSquaredError_1: 0.3169 - loss: 0.1755 - mse_loss
Epoch 23/100
4/4 ————— 0s 71ms/step - RootMeanSquaredError: 0.4226 - RootMeanSquaredError_1: 0.3146 - loss: 0.1710 - mse_loss
Epoch 24/100
4/4 ————— 0s 63ms/step - RootMeanSquaredError: 0.4171 - RootMeanSquaredError_1: 0.3123 - loss: 0.1666 - mse_loss
Epoch 25/100
4/4 ————— 0s 78ms/step - RootMeanSquaredError: 0.4116 - RootMeanSquaredError_1: 0.3102 - loss: 0.1624 - mse_loss
Epoch 26/100
4/4 ————— 0s 72ms/step - RootMeanSquaredError: 0.4062 - RootMeanSquaredError_1: 0.3081 - loss: 0.1584 - mse_loss
Epoch 27/100
4/4 ————— 0s 69ms/step - RootMeanSquaredError: 0.4010 - RootMeanSquaredError_1: 0.3061 - loss: 0.1544 - mse_loss
Epoch 28/100
4/4 ————— 0s 62ms/step - RootMeanSquaredError: 0.3958 - RootMeanSquaredError_1: 0.3042 - loss: 0.1506 - mse_loss
Epoch 29/100
4/4 ————— 0s 70ms/step - RootMeanSquaredError: 0.3906 - RootMeanSquaredError_1: 0.3023 - loss: 0.1468 - mse_loss
```

```
class PrintValTrainRatioCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs):
        ratio = logs["val_loss"] / logs["loss"]
        print(f"Epoch={epoch}, val/train={ratio:.2f}")
```

```
val_train_ratio_cb = PrintValTrainRatioCallback()
history = model.fit(
    (X_train_wide, X_train_deep), (y_train, y_train), epochs=10,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid)),
    callbacks=[val_train_ratio_cb], verbose=0)
```

```
Epoch=0, val/train=3.35
Epoch=1, val/train=3.38
Epoch=2, val/train=3.41
Epoch=3, val/train=3.44
Epoch=4, val/train=3.47
Epoch=5, val/train=3.50
Epoch=6, val/train=3.53
Epoch=7, val/train=3.55
```

```
Epoch=8, val/train=3.58
Epoch=9, val/train=3.61
```

▼ Sử dụng TensorBoard để Trực quan hóa

TensorBoard đã được cài đặt sẵn trên Colab, nhưng `tensorboard-plugin-profile` thì chưa, vì vậy hãy cài đặt nó:

```
if "google.colab" in sys.modules: # extra code
    %pip install -q -U tensorboard-plugin-profile
```

```
===== 14.9/14.9 MB 51.4 MB/s eta 0:00:00
===== 106.4/106.4 kB 6.5 MB/s eta 0:00:00
```

```
shutil.rmtree("my_logs", ignore_errors=True)
```

```
from pathlib import Path
from time import strftime

def get_run_logdir(root_logdir="my_logs"):
    return Path(root_logdir) / strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir()
```

```
# extra code - builds the first regression model we used earlier
tf.keras.backend.clear_session()
tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization(input_shape=X_train.shape[1:])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(30, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.SGD(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/preprocessing/tf_data_layer.py:19: UserWarning: Do not pass an `input_s
super().__init__(**kwargs)
```

```
import tensorflow as tf
import numpy as np
from pathlib import Path
from time import strftime

# 🛠 1. Giả lập dữ liệu đầu vào có cùng số mẫu
X_train = np.random.rand(1000, 8).astype("float32") # 1000 mẫu, 8 đặc trưng
y_train = np.random.rand(1000, 1).astype("float32") # ✅ 1000 mẫu tương ứng

X_valid = np.random.rand(200, 8).astype("float32")
y_valid = np.random.rand(200, 1).astype("float32")

# 🧠 2. Tạo model đơn giản
norm_layer = tf.keras.layers.Normalization(input_shape=[8])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(30, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
    tf.keras.layers.Dense(1)
])

optimizer = tf.keras.optimizers.SGD(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])

# ⚡ 3. Adapt normalization layer
norm_layer.adapt(X_train)

# 📊 4. Tạo thư mục log TensorBoard
def get_run_logdir(root_logdir="my_logs"):
    return Path(root_logdir) / strftime("run_%Y_%m_%d_%H_%M_%S")
```

```

run_logdir = get_run_logdir()

# 🕒 5. Khởi tạo callback TensorBoard
tensorboard_cb = tf.keras.callbacks.TensorBoard(
    run_logdir,
    profile_batch=(100, 200) # có thể bỏ nếu không cần profile
)

# 🐦 6. Huấn luyện model
history = model.fit(
    X_train, y_train,
    epochs=20,
    validation_data=(X_valid, y_valid),
    callbacks=[tensorboard_cb]
)

print("✅ Huấn luyện xong – Log TensorBoard đã được ghi tại:", run_logdir)

```

```

Epoch 1/20
32/32 ————— 2s 20ms/step - RootMeanSquaredError: 0.9026 - loss: 0.8189 - val_RootMeanSquaredError: 0.7528 - val_lo
Epoch 2/20
32/32 ————— 0s 9ms/step - RootMeanSquaredError: 0.6841 - loss: 0.4700 - val_RootMeanSquaredError: 0.5975 - val_lo
Epoch 3/20
32/32 ————— 0s 9ms/step - RootMeanSquaredError: 0.5511 - loss: 0.3047 - val_RootMeanSquaredError: 0.5024 - val_lo
Epoch 4/20
32/32 ————— 1s 9ms/step - RootMeanSquaredError: 0.4714 - loss: 0.2226 - val_RootMeanSquaredError: 0.4456 - val_lo
Epoch 5/20
32/32 ————— 0s 9ms/step - RootMeanSquaredError: 0.4251 - loss: 0.1809 - val_RootMeanSquaredError: 0.4120 - val_lo
Epoch 6/20
32/32 ————— 1s 6ms/step - RootMeanSquaredError: 0.3987 - loss: 0.1591 - val_RootMeanSquaredError: 0.3920 - val_lo
Epoch 7/20
32/32 ————— 0s 10ms/step - RootMeanSquaredError: 0.3835 - loss: 0.1471 - val_RootMeanSquaredError: 0.3795 - val_lo
Epoch 8/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3743 - loss: 0.1401 - val_RootMeanSquaredError: 0.3711 - val_lo
Epoch 9/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3683 - loss: 0.1357 - val_RootMeanSquaredError: 0.3650 - val_lo
Epoch 10/20
32/32 ————— 0s 6ms/step - RootMeanSquaredError: 0.3640 - loss: 0.1325 - val_RootMeanSquaredError: 0.3603 - val_lo
Epoch 11/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3607 - loss: 0.1301 - val_RootMeanSquaredError: 0.3563 - val_lo
Epoch 12/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3579 - loss: 0.1281 - val_RootMeanSquaredError: 0.3529 - val_lo
Epoch 13/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3555 - loss: 0.1264 - val_RootMeanSquaredError: 0.3498 - val_lo
Epoch 14/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3534 - loss: 0.1249 - val_RootMeanSquaredError: 0.3471 - val_lo
Epoch 15/20
32/32 ————— 0s 6ms/step - RootMeanSquaredError: 0.3514 - loss: 0.1235 - val_RootMeanSquaredError: 0.3445 - val_lo
Epoch 16/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3496 - loss: 0.1223 - val_RootMeanSquaredError: 0.3422 - val_lo
Epoch 17/20
32/32 ————— 0s 6ms/step - RootMeanSquaredError: 0.3480 - loss: 0.1211 - val_RootMeanSquaredError: 0.3400 - val_lo
Epoch 18/20
32/32 ————— 0s 5ms/step - RootMeanSquaredError: 0.3464 - loss: 0.1200 - val_RootMeanSquaredError: 0.3380 - val_lo
Epoch 19/20
32/32 ————— 0s 6ms/step - RootMeanSquaredError: 0.3449 - loss: 0.1190 - val_RootMeanSquaredError: 0.3361 - val_lo
Epoch 20/20
32/32 ————— 0s 6ms/step - RootMeanSquaredError: 0.3436 - loss: 0.1181 - val_RootMeanSquaredError: 0.3343 - val_lo
✅ Huấn luyện xong – Log TensorBoard đã được ghi tại: my_logs/run_2025_10_10_21_25_42

```

```

print("my_logs")
for path in sorted(Path("my_logs").glob("**/*")):
    print(" " * (len(path.parts) - 1) + path.parts[-1])

```

```

my_logs
run_2025_10_10_21_25_42
  train
    events.out.tfevents.1760131542.4e59ace1aa9e.366.0.v2
    plugins
    profile
      2025_10_10_21_25_47
      4e59ace1aa9e.xplane.pb
  validation
    events.out.tfevents.1760131544.4e59ace1aa9e.366.1.v2

```

Chúng ta hãy tải tiện ích mở rộng Jupyter `tensorboard` và khởi động máy chủ TensorBoard:

```
%load_ext tensorboard
%tensorboard --logdir=./my_logs
```

Lưu ý: nếu bạn muốn truy cập TensorBoard trong một tab riêng, hãy nhấp vào liên kết "localhost:6006" bên dưới:

```
# extra code

if "google.colab" in sys.modules:
    from google.colab import output

    output.serve_kernel_port_as_window(6006)
else:
    from IPython.display import display, HTML

    display(HTML('<a href="http://localhost:6006/">http://localhost:6006/</a>'))
```

Bạn cũng có thể trực quan hóa biểu đồ tần suất (histograms), hình ảnh, văn bản, và thậm chí nghe âm thanh bằng cách sử dụng TensorBoard:

```
test_logdir = get_run_logdir()
writer = tf.summary.create_file_writer(str(test_logdir))
with writer.as_default():
    for step in range(1, 1000 + 1):
        tf.summary.scalar("my_scalar", np.sin(step / 10), step=step)

        data = (np.random.randn(100) + 2) * step / 100 # gets larger
        tf.summary.histogram("my_hist", data, buckets=50, step=step)
```



```

images = np.random.rand(2, 32, 32, 3) * step / 1000 # gets brighter
tf.summary.image("my_images", images, step=step)

texts = ["The step is " + str(step), "Its square is " + str(step ** 2)]
tf.summary.text("my_text", texts, step=step)

sine_wave = tf.math.sin(tf.range(12000) / 48000 * 2 * np.pi * step)
audio = tf.reshape(tf.cast(sine_wave, tf.float32), [1, -1, 1])
tf.summary.audio("my_audio", audio, sample_rate=48000, step=step)

```

Lưu ý: trước đây có thể dễ dàng chia sẻ nhật ký TensorBoard của bạn với mọi người bằng cách tải chúng lên <https://tensorboard.dev/>. Đáng buồn thay, dịch vụ này sẽ ngừng hoạt động vào tháng 12 năm 2023, vì vậy tôi đã xóa các ví dụ mã tương ứng khỏi sổ tay (notebook) này.

Khi bạn dừng **nhân Jupyter** (Jupyter kernel) này (còn gọi là **Runtime**), nó cũng sẽ tự động dừng máy chủ TensorBoard. Một cách khác để dừng máy chủ TensorBoard là **kill** nó, nếu bạn đang chạy trên Linux hoặc MacOSX. Đầu tiên, bạn cần tìm **ID tiến trình** (process ID) của nó:

```

# extra code - lists all running TensorBoard server instances

from tensorboard import notebook

notebook.list()

Known TensorBoard instances:
- port 6006: logdir ./my_logs (started 0:00:43 ago; pid 19019)

```

Tiếp theo, bạn có thể sử dụng lệnh sau trên Linux hoặc MacOSX, thay thế `<pid>` bằng pid được liệt kê ở trên:

```
!kill <pid>
```

Trên Windows:

```
!taskkill /F /PID <pid>
```

✓ Tinh chỉnh Siêu tham số của Mạng Nơ-ron

Trong phần này, chúng ta sẽ sử dụng lại tập dữ liệu **Fashion MNIST**:

```

(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]

```

```

tf.keras.backend.clear_session()
tf.random.set_seed(42)

```

```

if "google.colab" in sys.modules:
    %pip install -q -U keras_tuner

```

129.1/129.1 kB 5.1 MB/s eta 0:00:00

```

import keras_tuner as kt

def build_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    n_neurons = hp.Int("n_neurons", min_value=16, max_value=256)
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,
                              sampling="log")
    optimizer = hp.Choice("optimizer", values=["sgd", "adam"])
    if optimizer == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    else:
        optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)

```

```

model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten())
for _ in range(n_hidden):
    model.add(tf.keras.layers.Dense(n_neurons, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
return model

```

```

random_search_tuner = kt.RandomSearch(
    build_model, objective="val_accuracy", max_trials=5, overwrite=True,
    directory="my_fashion_mnist", project_name="my_rnd_search", seed=42)
random_search_tuner.search(X_train, y_train, epochs=10,
                          validation_data=(X_valid, y_valid))

```

```

Trial 5 Complete [00h 01m 13s]
val_accuracy: 0.8370000123977661

```

```

Best val_accuracy So Far: 0.8500000238418579
Total elapsed time: 00h 06m 18s

```

```

top3_models = random_search_tuner.get_best_models(num_models=3)
best_model = top3_models[0]

```

```

top3_params = random_search_tuner.get_best_hyperparameters(num_trials=3)
top3_params[0].values # best hyperparameter values

```

```

{'n_hidden': 7,
 'n_neurons': 100,
 'learning_rate': 0.0012482904754698163,
 'optimizer': 'sgd'}

```

```

best_trial = random_search_tuner.oracle.get_best_trials(num_trials=1)[0]
best_trial.summary()

```

```

Trial 1 summary
Hyperparameters:
n_hidden: 7
n_neurons: 100
learning_rate: 0.0012482904754698163
optimizer: sgd
Score: 0.8500000238418579

```

```

best_trial.metrics.get_last_value("val_accuracy")

```

```

np.float64(0.8500000238418579)

```

```

best_model.fit(X_train_full, y_train_full, epochs=10)
test_loss, test_accuracy = best_model.evaluate(X_test, y_test)

```

```

Epoch 1/10
1875/1875 ————— 9s 4ms/step - accuracy: 0.8750 - loss: 0.3357
Epoch 2/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.8789 - loss: 0.3228
Epoch 3/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.8843 - loss: 0.3120
Epoch 4/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.8875 - loss: 0.3027
Epoch 5/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.8911 - loss: 0.2942
Epoch 6/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.8933 - loss: 0.2859
Epoch 7/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.8951 - loss: 0.2800
Epoch 8/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.8986 - loss: 0.2739
Epoch 9/10
1875/1875 ————— 7s 3ms/step - accuracy: 0.9001 - loss: 0.2676
Epoch 10/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9026 - loss: 0.2619
313/313 ————— 1s 2ms/step - accuracy: 0.8555 - loss: 0.4183

```

```

class MyClassificationHyperModel(kt.HyperModel):
    def build(self, hp):
        return build_model(hp)

```

```
def fit(self, hp, model, X, y, **kwargs):
    if hp.Boolean("normalize"):
        norm_layer = tf.keras.layers.Normalization()
        X = norm_layer(X)
    return model.fit(X, y, **kwargs)
```

```
hyperband_tuner = kt.Hyperband(
    MyClassificationHyperModel(), objective="val_accuracy", seed=42,
    max_epochs=10, factor=3, hyperband_iterations=2,
    overwrite=True, directory="my_fashion_mnist", project_name="hyperband")
```

```
root_logdir = Path(hyperband_tuner.project_dir) / "tensorboard"
tensorboard_cb = tf.keras.callbacks.TensorBoard(root_logdir)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=2)
hyperband_tuner.search(X_train, y_train, epochs=10,
                       validation_data=(X_valid, y_valid),
                       callbacks=[early_stopping_cb, tensorboard_cb])
```

```
Trial 49 Complete [00h 01m 10s]
val_accuracy: 0.8515999913215637
```

```
Best val_accuracy So Far: 0.8700000047683716
Total elapsed time: 00h 25m 36s
```

```
Search: Running Trial #50
```

Value	Best Value So Far	Hyperparameter
7	7	n_hidden
247	100	n_neurons
0.00039877	0.00044489	learning_rate
adam	adam	optimizer
True	False	normalize
4	10	tuner/epochs
0	4	tuner/initial_epoch
1	2	tuner/bracket
0	2	tuner/round

```
Epoch 1/4
795/1719 ————— 15s 17ms/step - accuracy: 0.6817 - loss: 1.5682
```

```
bayesian_opt_tuner = kt.BayesianOptimization(
    MyClassificationHyperModel(), objective="val_accuracy", seed=42,
    max_trials=10, alpha=1e-4, beta=2.6,
    overwrite=True, directory="my_fashion_mnist", project_name="bayesian_opt")
bayesian_opt_tuner.search(X_train, y_train, epochs=10,
                          validation_data=(X_valid, y_valid),
                          callbacks=[early_stopping_cb])
```

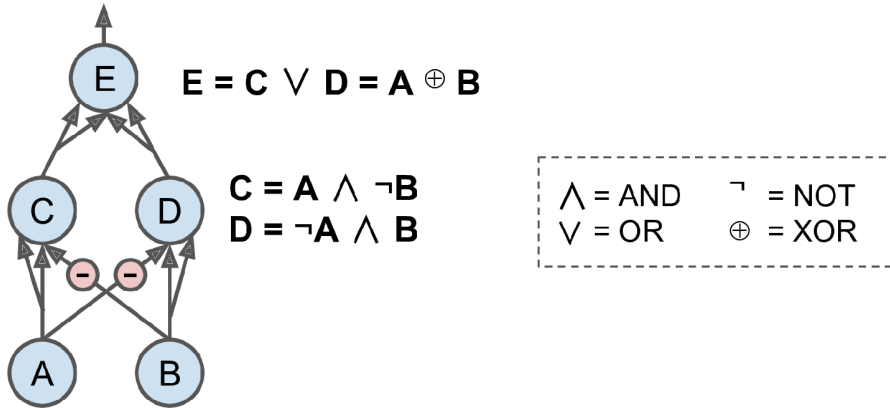
```
%tensorboard --logdir {root_logdir}
```

✧ Hướng dẫn bài tập

✧ 1. tới 9.

1. Truy cập [TensorFlow Playground](#) và thử nghiệm với nó, như đã mô tả trong bài tập này.
2. Đây là một mạng nơ-ron dựa trên các nơ-ron nhân tạo nguyên bản tính toán $A \oplus B$ (trong đó $\oplus$ đại diện cho phép XOR (exclusive OR)), sử dụng thực tế là $A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$. Có những giải pháp khác—ví dụ, sử dụng thực tế là $A \oplus B = (A \vee B) \wedge \neg(A \wedge B)$,

hoặc thực tế là $A \oplus B = (A \vee B) \wedge (\neg A \vee \neg B)$, v.v.



- Một Perceptron cổ điển sẽ hội tụ chỉ khi tập dữ liệu có thể được **phân tách tuyến tính** (linearly separable), và nó sẽ không thể ước tính **xác suất lớp** (class probabilities). Ngược lại, một bộ phân loại **Hồi quy Logistic** (Logistic Regression) thường sẽ hội tụ đến một giải pháp hợp lý tốt ngay cả khi tập dữ liệu không thể phân tách tuyến tính, và nó sẽ xuất ra xác suất lớp. Nếu bạn thay đổi hàm kích hoạt của Perceptron thành **hàm kích hoạt sigmoid** (hoặc hàm kích hoạt softmax nếu có nhiều nơ-ron), và nếu bạn huấn luyện nó bằng cách sử dụng **Gradient Descent** (hoặc một thuật toán tối ưu hóa khác nhằm giảm thiểu hàm chi phí, thường là **cross entropy**), thì nó sẽ trở nên tương đương với một bộ phân loại Hồi quy Logistic.
- Hàm kích hoạt sigmoid** là một thành phần quan trọng trong việc huấn luyện các MLP đầu tiên vì **đạo hàm** của nó luôn khác 0, do đó Gradient Descent luôn có thể lần xuống dốc. Khi hàm kích hoạt là **hàm bước** (step function), Gradient Descent không thể di chuyển, vì hoàn toàn không có độ dốc nào cả.
- Các hàm kích hoạt phổ biến bao gồm hàm bước, hàm sigmoid, **hàm hyperbolic tangent (tanh)**, và **hàm Đơn vị Tuyến tính Chính lưu (ReLU)** (xem Hình 10-8). Xem Chương 11 để biết các ví dụ khác, chẳng hạn như ELU và các biến thể của hàm ReLU.
- Xét MLP được mô tả trong câu hỏi, bao gồm một lớp đầu vào với 10 nơ-ron truyền qua (passthrough neurons), tiếp theo là một lớp ẩn với 50 nơ-ron nhân tạo, và cuối cùng là một lớp đầu ra với 3 nơ-ron nhân tạo, trong đó tất cả các nơ-ron nhân tạo đều sử dụng hàm kích hoạt ReLU:
 - Hình dạng** (shape) của **ma trận đầu vào X** là $m \times 10$, trong đó m đại diện cho **kích thước lô huấn luyện** (training batch size).
 - Hình dạng của **ma trận trọng số** (weight matrix) $\mathbf{W}_h$ của lớp ẩn là 10×50 , và **độ dài** của **vector độ lệch** (bias vector) $\mathbf{b}_h$ của nó là 50.
 - Hình dạng của ma trận trọng số $\mathbf{W}_o$ của lớp đầu ra là 50×3 , và độ dài của vector độ lệch $\mathbf{b}_o$ của nó là 3.
 - Hình dạng của **ma trận đầu ra** (output matrix) $\mathbf{Y}$ của mạng là $m \times 3$.
 - $\mathbf{Y} = \text{ReLU}(\text{ReLU}(\mathbf{X} \mathbf{W}_h + \mathbf{b}_h) \mathbf{W}_o + \mathbf{b}_o)$. Nhớ lại rằng hàm ReLU chỉ đặt mọi số âm trong ma trận về 0. Cũng lưu ý rằng khi bạn cộng một vector độ lệch vào một ma trận, nó được cộng vào mọi hàng trong ma trận, được gọi là **phát sóng** (broadcasting).
- Để phân loại email thành **spam** hoặc **ham** (không phải spam), bạn chỉ cần một nơ-ron trong lớp đầu ra của mạng nơ-ron—ví dụ, cho biết xác suất email đó là spam. Bạn thường sử dụng **hàm kích hoạt sigmoid** trong lớp đầu ra khi ước tính một xác suất. Nếu thay vào đó bạn muốn giải quyết bài toán MNIST, bạn cần 10 nơ-ron trong lớp đầu ra và bạn phải thay thế hàm sigmoid bằng **hàm kích hoạt softmax**, có thể xử lý nhiều lớp, xuất ra một xác suất cho mỗi lớp. Nếu bạn muốn mạng nơ-ron của mình dự đoán giá nhà như trong Chương 2, thì bạn cần một nơ-ron đầu ra, không sử dụng bất kỳ hàm kích hoạt nào trong lớp đầu ra. Lưu ý: khi các giá trị cần dự đoán có thể thay đổi theo nhiều **bậc độ lớn** (orders of magnitude), bạn có thể muốn dự đoán **logarit** của giá trị mục tiêu thay vì giá trị mục tiêu trực tiếp. Đơn giản chỉ cần tính **hàm mũ** (exponential) của đầu ra mạng nơ-ron sẽ cho bạn giá trị ước tính (vì $\exp(\log v) = v$).
- Backpropagation** là một kỹ thuật được sử dụng để huấn luyện mạng nơ-ron nhân tạo. Đầu tiên nó tính toán **gradient** của hàm chi phí đối với mọi tham số mô hình (tất cả các trọng số và độ lệch), sau đó nó thực hiện một bước Gradient Descent sử dụng các gradient này. Bước backpropagation này thường được thực hiện hàng ngàn hoặc hàng triệu lần, sử dụng nhiều **lô huấn luyện** (training batches), cho đến khi các tham số mô hình hội tụ đến các giá trị mà (hy vọng) giảm thiểu hàm chi phí. Để tính toán các gradient, backpropagation sử dụng **autodiff chế độ đảo ngược** (reverse-mode autodiff) (mặc dù nó không được gọi như vậy khi backpropagation được phát minh, và nó đã được tái phát minh nhiều lần). Autodiff chế độ đảo ngược thực hiện một **lượt chuyển tiếp** (forward pass) qua **đồ thị tính toán** (computation graph), tính toán giá trị của mọi nút cho lô huấn luyện hiện tại, và sau đó nó thực hiện một **lượt đảo ngược** (reverse pass), tính toán tất cả các gradient cùng một lúc (xem Phụ lục B để biết thêm chi tiết). Vậy sự khác biệt là gì? Backpropagation đề cập đến toàn bộ quá trình huấn luyện một mạng nơ-ron nhân tạo bằng cách sử dụng nhiều bước backpropagation, mỗi bước tính toán gradient và sử dụng chúng để thực hiện một bước Gradient Descent. Ngược lại, autodiff chế độ đảo ngược chỉ là một kỹ thuật để tính toán gradient một cách hiệu quả, và nó tình cờ được sử dụng bởi backpropagation.
- Dưới đây là danh sách tất cả các **siêu tham số** (hyperparameters) bạn có thể tinh chỉnh trong một MLP cơ bản: **số lượng lớp ẩn** (number of hidden layers), **số lượng nơ-ron** trong mỗi lớp ẩn, và **hàm kích hoạt** được sử dụng trong mỗi lớp ẩn và trong lớp đầu ra. Nói chung, **hàm kích hoạt ReLU** (hoặc một trong các biến thể của nó; xem Chương 11) là một mặc định tốt cho các lớp ẩn. Đối với

lớp đầu ra, nói chung bạn sẽ muốn **hàm kích hoạt sigmoid** cho **phân loại nhị phân** (binary classification), **hàm kích hoạt softmax** cho **phân loại đa lớp** (multiclass classification), hoặc **không có hàm kích hoạt** cho **hồi quy** (regression). Nếu MLP bị **quá khớp** (overfits) dữ liệu huấn luyện, bạn có thể thử giảm số lượng lớp ẩn và giảm số lượng nơ-ron trên mỗi lớp ẩn.

10.

Bài tập: Huấn luyện một **MLP sâu** trên tập dữ liệu **MNIST** (bạn có thể tải nó bằng cách sử dụng `tf.keras.datasets.mnist.load_data()`). Xem liệu bạn có thể đạt được độ chính xác trên 98% bằng cách **tinh chỉnh thủ công** (manually tuning) các **siêu tham số** (hyperparameters) hay không. Hãy thử tìm kiếm **tốc độ học tối ưu** (optimal learning rate) bằng cách sử dụng phương pháp được trình bày trong chương này (tức là, bằng cách tăng tốc độ học theo cấp số nhân, vẽ đồ thị hàm mất mát, và tìm điểm mà tại đó hàm mất mát tăng vọt). Tiếp theo, hãy thử tinh chỉnh các siêu tham số bằng cách sử dụng **Keras Tuner** với đầy đủ các tính năng—lưu **điểm kiểm tra** (checkpoints), sử dụng **dừng sớm** (early stopping), và vẽ đồ thị **đường cong học tập** (learning curves) bằng **TensorBoard**.

TODO: cập nhật giải pháp này để sử dụng Keras Tuner.

Chúng ta hãy tải tập dữ liệu:

```
(X_train_full, y_train_full), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
```

Giống như đối với tập dữ liệu Fashion MNIST, tập huấn luyện MNIST chứa 60.000 hình ảnh thang độ xám (grayscale), mỗi hình có kích thước 28x28 pixel:

```
X_train_full.shape
```

Cường độ của mỗi pixel cũng được biểu diễn dưới dạng một byte (từ 0 đến 255):

```
X_train_full.dtype
```

Chúng ta hãy chia tập huấn luyện đầy đủ thành một **tập xác thực** (validation set) và một **tập huấn luyện** (training set) (nhỏ hơn). Chúng ta cũng **chia tỷ lệ** (scale) cường độ pixel xuống phạm vi từ 0 đến 1 và chuyển chúng thành kiểu **số thực** (float), bằng cách chia cho 255, giống như chúng ta đã làm với Fashion MNIST:

```
X_valid, X_train = X_train_full[:5000] / 255., X_train_full[5000:] / 255.
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
X_test = X_test / 255.
```

Chúng ta hãy vẽ một hình ảnh bằng cách sử dụng hàm `imshow()` của Matplotlib, với bản đồ màu (color map) là `'binary'`:

```
plt.imshow(X_train[0], cmap="binary")
plt.axis('off')
plt.show()
```

Các nhãn là **ID lớp** (class ID) (được biểu diễn dưới dạng uint8), từ 0 đến 9. Thuận tiện, các ID lớp tương ứng với các chữ số được biểu diễn trong hình ảnh, vì vậy chúng ta không cần một mảng `class_names`:

```
y_train
```

Tập xác thực (validation set) chứa 5.000 hình ảnh, và tập kiểm tra (test set) chứa 10.000 hình ảnh:

```
X_valid.shape
```

```
X_test.shape
```

Chúng ta hãy xem một mẫu các hình ảnh trong tập dữ liệu:

```

n_rows = 4
n_cols = 10
plt.figure(figsize=(n_cols * 1.2, n_rows * 1.2))
for row in range(n_rows):
    for col in range(n_cols):
        index = n_cols * row + col
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(X_train[index], cmap="binary", interpolation="nearest")
        plt.axis('off')
        plt.title(y_train[index])
plt.subplots_adjust(wspace=0.2, hspace=0.5)
plt.show()

```

Chúng ta hãy xây dựng một **mạng dày đặc** (dense network) đơn giản và tìm **tốc độ học tối ưu** (optimal learning rate). Chúng ta sẽ cần một **callback** để tăng tốc độ học ở mỗi lần lặp. Nó cũng sẽ ghi lại tốc độ học và **hàm mất mát** (loss) ở mỗi lần lặp:

```

K = tf.keras.backend

class ExponentialLearningRate(tf.keras.callbacks.Callback):
    def __init__(self, factor):
        self.factor = factor
        self.rates = []
        self.losses = []
    def on_batch_end(self, batch, logs):
        self.rates.append(K.get_value(self.model.optimizer.learning_rate))
        self.losses.append(logs["loss"])
        K.set_value(self.model.optimizer.learning_rate, self.model.optimizer.learning_rate * self.factor)

```

```

tf.keras.backend.clear_session()
np.random.seed(42)
tf.random.set_seed(42)

```

```

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

```

Chúng ta sẽ bắt đầu với một **tốc độ học** (learning rate) nhỏ là $1e-3$, và tăng nó thêm 0.5% ở mỗi lần lặp:

```

optimizer = tf.keras.optimizers.SGD(learning_rate=1e-3)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
expon_lr = ExponentialLearningRate(factor=1.005)

```

Bây giờ chúng ta hãy huấn luyện mô hình chỉ trong 1 **epoch**:

```

history = model.fit(X_train, y_train, epochs=1,
                    validation_data=(X_valid, y_valid),
                    callbacks=[expon_lr])

```

Bây giờ chúng ta có thể vẽ đồ thị **hàm mất mát** (loss) như là một **hàm số** của **tốc độ học** (learning rate):

```

plt.plot(expon_lr.rates, expon_lr.losses)
plt.gca().set_xscale('log')
plt.hlines(min(expon_lr.losses), min(expon_lr.rates), max(expon_lr.rates))
plt.axis([min(expon_lr.rates), max(expon_lr.rates), 0, expon_lr.losses[0]])
plt.grid()
plt.xlabel("Learning rate")
plt.ylabel("Loss")

```

Hàm mất mát (loss) bắt đầu tăng vọt mạnh mẽ trở lại khi **tốc độ học** (learning rate) vượt quá $6e-1$, vì vậy chúng ta hãy thử sử dụng một nửa giá trị đó, là $3e-1$:

```

tf.keras.backend.clear_session()
np.random.seed(42)

```

```
tf.random.set_seed(42)
```

```
model = tf.keras.Sequential([  
    tf.keras.layers.Flatten(input_shape=[28, 28]),  
    tf.keras.layers.Dense(300, activation="relu"),  
    tf.keras.layers.Dense(100, activation="relu"),  
    tf.keras.layers.Dense(10, activation="softmax")  
])
```

```
optimizer = tf.keras.optimizers.SGD(learning_rate=3e-1)  
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,  
              metrics=["accuracy"])
```

```
run_index = 1 # increment this at every run  
run_logdir = Path() / "my_mnist_logs" / "run_{:03d}".format(run_index)  
run_logdir
```

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=20)  
checkpoint_cb = tf.keras.callbacks.ModelCheckpoint("my_mnist_model", save_best_only=True)  
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir)  
  
history = model.fit(X_train, y_train, epochs=100,  
                   validation_data=(X_valid, y_valid),  
                   callbacks=[checkpoint_cb, early_stopping_cb, tensorboard_cb])
```

```
model = tf.keras.models.load_model("my_mnist_model") # rollback to best model  
model.evaluate(X_test, y_test)
```